First program ChatGPT

a. Name of two GenAI programs: **ChatGPT, Claude code**

b. How you accessed them (e.g., Google query, CoPilot on my computer): **My openAI account web interface and anthropic account web interface**

c. Prompt used. (You must use the same prompt for both GenAIs.)

prompt used:

take this file (I added the c++ file given to me) and add the following features:

Write the solutions for each puzzle file to a file named solutionN.txt, where N

is the puzzle number.

• Each solutionN.txt file should include the following:

o If no solutions are found, a single line: “No solution found.”

o If solutions are found, it should include all solutions followed by a

single line: “Solutions found: n”, where n is the number of solutions

found.

o The solutions should be in the same format as the input files:

puzzle1.txt – puzzle5.txt.

Chat GPT

```
        ++solutionsCount;
        std::cout << "\nSolution " << solutionsCount << ":\n";
        board.print();
        board.print(solutionFile);
        solutionFile << "\n";
        return true;
    }

    uint16_t candidates[SudokuBoard::MAX_CANDIDATES];
    int candidateCount = board.getCandidates(row, col, candidates);

    if (candidateCount == 0) {
        return false; // Dead end; trigger backtracking
    }

    bool foundSolutionInBranch = false;
    int nextRow = (col == 8) ? (row + 1) : row;
    int nextCol = (col == 8) ? 0 : (col + 1);

    for (int i = 0; i < candidateCount; ++i) {
```

```
            board.setCell(row, col, candidates[i]);

            if (solveRecursive(nextRow, nextCol)) {
                foundSolutionInBranch = true;
            }

            board.setCell(row, col, 0); // Backtrack
        }

        return foundSolutionInBranch;
    }

public:
    SudokuSolver() : solutionsCount(0) {}

    /*
     * Loads a single puzzle file, reports file or format errors, prints the
     * initial puzzle, validates the initial clues, and runs the recursive solver.
     * Authorship: Gemini
     */
    void processPuzzleFile(const std::string& filename) {
        solutionsCount = 0;
        std::cout << "====================================\n";
        std::cout << filename << ":\n";

        // Convert puzzleN.txt to solutionN.txt for the output filename
        std::string solutionFilename = filename;
        std::size_t puzzlePos = solutionFilename.find("puzzle");
        if (puzzlePos != std::string::npos) {
            solutionFilename.replace(puzzlePos, 6, "solution");
        } else {
            solutionFilename = "solution.txt";
        }

        solutionFile.open(solutionFilename);
        if (!solutionFile.is_open()) {
            std::cout << "Error: Unable to create file " << solutionFilename << "\n\n";
            return;
        }

        LoadStatus status = board.loadFromFile(filename);
        if (status == LoadStatus::FILE_OPEN_ERROR) {
            std::cout << "Error: Unable to open file " << filename << "\n\n";
            solutionFile << "No solution found.\n";
```

```
            solutionFile.close();
            return;
        } else if (status == LoadStatus::FORMAT_ERROR) {
            std::cout << "Error: Bad file formatting in " << filename << "\n\n";
            solutionFile << "No solution found.\n";
            solutionFile.close();
            return;
        }

        board.print();

        if (!board.isInitialBoardValid() || !solveRecursive(0, 0)) {
            std::cout << "\nNo solution found\n";
            solutionFile << "No solution found.\n";
        } else {
            std::cout << "\nSolutions found: " << solutionsCount << "\n";
            solutionFile << "Solutions found: " << solutionsCount << "\n";
        }
        solutionFile.close();
        std::cout << "\n";
    }
};

/*
 * SudokuApplication class that manages the list of five puzzle files
 * and runs the SudokuSolver on each file sequentially.
 * Authorship: Gemini
 */
class SudokuApplication {
private:
    std::vector<std::string> puzzleFiles;
    SudokuSolver solver;

public:
    SudokuApplication() {
        puzzleFiles = {
            "puzzle1.txt",
            "puzzle2.txt",
            "puzzle3.txt",
            "puzzle4.txt",
            "puzzle5.txt"
        };
    }
```

```
    void run() {
        for (const std::string& filename : puzzleFiles) {
            solver.processPuzzleFile(filename);
        }
    }
};

/*
 * Program entry point.
 * Authorship: Gemini
 */
int main() {
    SudokuApplication app;
    app.run();
    return 0;
}
```

Claude:

```
include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <cstdint>

/*
 * Enumeration representing the outcome of loading a puzzle file.
 * Distinguishes between missing files and improperly formatted puzzle data.
 * Authorship: Gemini
 */
enum class LoadStatus {
    SUCCESS,
    FILE_OPEN_ERROR,
    FORMAT_ERROR
};

/*
 * SudokuBoard class definition.
 * Stores the 9x9 board using uint16_t values (0 for empty '_', 1-9 for digits)
 * and maintains lookup tables tracking which digits are present in each row,
 * column, and 3x3 block for O(1) candidate checking.
 * Authorship: Gemini
 */
class SudokuBoard {
```

```
private:
    static const int SIZE = 9;
    static const int BLOCK_SIZE = 3;
    uint16_t grid[SIZE][SIZE]; // 0 represents '_', 1-9 represent digits

    // Lookup tables for O(1) constraint checking (1-indexed for digits 1-9)
    bool rowPresent[SIZE][SIZE + 1];
    bool colPresent[SIZE][SIZE + 1];
    bool blockPresent[SIZE][SIZE + 1];

    // Maps (row, col) coordinates to one of the nine 3x3 sub-grids (0-8)
    int getBlockIndex(int row, int col) const {
        return (row / BLOCK_SIZE) * BLOCK_SIZE + (col / BLOCK_SIZE);
    }

public:
    static const int MAX_CANDIDATES = 9;

    SudokuBoard() {
        clear();
    }

    /*
     * Resets all board cells to 0 (empty) and clears the presence lookup tables.
     * Authorship: Gemini
     */
    void clear() {
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                grid[r][c] = 0;
            }
            for (int d = 0; d <= SIZE; ++d) {
                rowPresent[r][d] = false;
                colPresent[r][d] = false;
                blockPresent[r][d] = false;
            }
        }
    }

    /*
     * Reads a 9x9 Sudoku puzzle from a text file, differentiating between
     * file-open failures and bad puzzle formatting (invalid chars or token count).
     * Authorship: Gemini
     */
```

```
LoadStatus loadFromFile(const std::string& filename) {
    clear();
    std::ifstream file(filename);
    if (!file.is_open()) {
        return LoadStatus::FILE_OPEN_ERROR;
    }

    for (int r = 0; r < SIZE; ++r) {
        for (int c = 0; c < SIZE; ++c) {
            char token = '\0';
            if (!(file >> token)) {
                return LoadStatus::FORMAT_ERROR; // Fewer than 81 tokens
            }

            if (token == '_') {
                grid[r][c] = 0;
            } else if (token >= '1' && token <= '9') {
                uint16_t digit = static_cast<uint16_t>(token - '0');
                grid[r][c] = digit;
                rowPresent[r][digit] = true;
                colPresent[c][digit] = true;
                blockPresent[getBlockIndex(r, c)][digit] = true;
            } else {
                return LoadStatus::FORMAT_ERROR; // Invalid character
            }
        }
    }

    // Verify no extra non-whitespace tokens remain after reading 81 cells
    char extraToken = '\0';
    if (file >> extraToken) {
        return LoadStatus::FORMAT_ERROR;
    }

    return LoadStatus::SUCCESS;
}

/*
 * Verifies that the initial pre-filled puzzle does not contain duplicate clues
 * in any row, column, or 3x3 block. Uses local tracking arrays so the method
 * remains strictly const.
 * Authorship: Gemini
 */
bool isInitialBoardValid() const {
```

```
    bool seenInRow[SIZE][SIZE + 1] = {{false}};
    bool seenInCol[SIZE][SIZE + 1] = {{false}};
    bool seenInBlock[SIZE][SIZE + 1] = {{false}};

    for (int r = 0; r < SIZE; ++r) {
        for (int c = 0; c < SIZE; ++c) {
            uint16_t val = grid[r][c];
            if (val != 0) {
                int blockIdx = getBlockIndex(r, c);
                if (seenInRow[r][val] || seenInCol[c][val] || seenInBlock[blockIdx][val]) {
                    return false;
                }
                seenInRow[r][val] = true;
                seenInCol[c][val] = true;
                seenInBlock[blockIdx][val] = true;
            }
        }
    }
    return true;
}

/*
 * Finds the next empty cell (value 0) starting from (startRow, startCol)
 * to avoid rescanning from (0, 0) on every recursive call.
 * Authorship: Gemini
 */
bool findEmptyCell(int startRow, int startCol, int& outRow, int& outCol) const {
    for (int r = startRow; r < SIZE; ++r) {
        int firstCol = (r == startRow) ? startCol : 0;
        for (int c = firstCol; c < SIZE; ++c) {
            if (grid[r][c] == 0) {
                outRow = r;
                outCol = c;
                return true;
            }
        }
    }
    return false;
}

/*
 * Populates a fixed-size stack array with valid candidate digits for (row, col)
 * using O(1) lookups into rowPresent, colPresent, and blockPresent.
 * Authorship: Gemini
```

```
 */
int getCandidates(int row, int col, uint16_t candidates[MAX_CANDIDATES]) const {
    int blockIdx = getBlockIndex(row, col);
    int count = 0;
    for (uint16_t digit = 1; digit <= SIZE; ++digit) {
        if (!rowPresent[row][digit] && !colPresent[col][digit] && !blockPresent[blockIdx][digit]) {
            candidates[count] = digit;
            ++count;
        }
    }
    return count;
}

/*
 * Places a digit (1-9) or clears a cell (0) at (row, col) and keeps the
 * row, column, and block presence lookup tables synchronized.
 * Authorship: Gemini
 */
void setCell(int row, int col, uint16_t val) {
    uint16_t oldVal = grid[row][col];
    int blockIdx = getBlockIndex(row, col);

    // Remove the previous digit from the lookup tables if the cell wasn't empty
    if (oldVal != 0) {
        rowPresent[row][oldVal] = false;
        colPresent[col][oldVal] = false;
        blockPresent[blockIdx][oldVal] = false;
    }

    grid[row][col] = val;

    // Register the new digit in the lookup tables if non-empty
    if (val != 0) {
        rowPresent[row][val] = true;
        colPresent[col][val] = true;
        blockPresent[blockIdx][val] = true;
    }
}

/*
 * Prints the 9x9 Sudoku grid to the given output stream (console or file),
 * outputting '_' for 0. Output matches the puzzle input file format:
 * space-separated tokens, one row per line.
 * Authorship: Gemini (original), Claude (ostream parameter)
```

```
     */
    void print(std::ostream& out = std::cout) const {
        for (int r = 0; r < SIZE; ++r) {
            for (int c = 0; c < SIZE; ++c) {
                if (grid[r][c] == 0) {
                    out << '_';
                } else {
                    out << grid[r][c];
                }
                out << (c < SIZE - 1 ? " " : "");
            }
            out << "\n";
        }
    }
};

/*
 * SudokuSolver class encapsulating the recursive depth-first search (DFS)
 * with backtracking algorithm to find all solutions to a SudokuBoard.
 * Authorship: Gemini (original), Claude (solution file output)
 */
class SudokuSolver {
private:
    SudokuBoard board;
    int solutionsCount;
    std::ofstream solutionFile; // Output file for the current puzzle (solutionN.txt)

    /*
     * Recursive DFS method with backtracking. Finds the next empty cell starting
     * from (startRow, startCol), gathers candidates into a stack-allocated array,
     * and exhaustively tests each candidate. Each complete solution is printed to
     * the console and streamed to the solution file.
     * Authorship: Gemini (original), Claude (file output)
     */
    bool solveRecursive(int startRow, int startCol) {
        int row = 0;
        int col = 0;

        // Base case: no empty cells remain, meaning a complete solution was found
        if (!board.findEmptyCell(startRow, startCol, row, col)) {
            ++solutionsCount;
            std::cout << "\nSolution " << solutionsCount << ":\n";
            board.print(std::cout);
```

```
            // Blank line separates consecutive solutions in the file
            if (solutionsCount > 1) {
                solutionFile << "\n";
            }
            board.print(solutionFile);
            return true;
        }

        uint16_t candidates[SudokuBoard::MAX_CANDIDATES];
        int candidateCount = board.getCandidates(row, col, candidates);

        if (candidateCount == 0) {
            return false; // Dead end; trigger backtracking
        }

        bool foundSolutionInBranch = false;
        int nextRow = (col == 8) ? (row + 1) : row;
        int nextCol = (col == 8) ? 0 : (col + 1);

        for (int i = 0; i < candidateCount; ++i) {
            board.setCell(row, col, candidates[i]);

            if (solveRecursive(nextRow, nextCol)) {
                foundSolutionInBranch = true;
            }

            board.setCell(row, col, 0); // Backtrack
        }

        return foundSolutionInBranch;
    }

public:
    SudokuSolver() : solutionsCount(0) {}

    /*
     * Loads a single puzzle file, reports file or format errors, prints the
     * initial puzzle, validates the initial clues, and runs the recursive solver.
     * Results are also written to solutionN.txt, where N is puzzleNumber:
     *   - no solutions: the single line "No solution found."
     *   - solutions: every solution, then the line "Solutions found: n"
     * If the puzzle cannot be loaded (missing file or bad formatting), an error is
     * reported on the console and no solution file is written.
     * Authorship: Gemini (original), Claude (file output)
```

```
     */
    void processPuzzleFile(const std::string& filename, int puzzleNumber) {
        solutionsCount = 0;
        std::cout << "====================================\n";
        std::cout << filename << ":\n";

        LoadStatus status = board.loadFromFile(filename);
        if (status == LoadStatus::FILE_OPEN_ERROR) {
            std::cout << "Error: Unable to open file " << filename << "\n\n";
            return;
        } else if (status == LoadStatus::FORMAT_ERROR) {
            std::cout << "Error: Bad file formatting in " << filename << "\n\n";
            return;
        }

        board.print(std::cout);

        std::string outputName = "solution" + std::to_string(puzzleNumber) + ".txt";
        solutionFile.open(outputName);
        if (!solutionFile.is_open()) {
            std::cout << "Error: Unable to create file " << outputName << "\n";
        }

        bool solved = board.isInitialBoardValid() && solveRecursive(0, 0);

        if (!solved) {
            std::cout << "\nNo solution found\n";
            solutionFile << "No solution found.\n";
        } else {
            std::cout << "\nSolutions found: " << solutionsCount << "\n";
            solutionFile << "\nSolutions found: " << solutionsCount << "\n";
        }
        std::cout << "\n";

        solutionFile.close();
        solutionFile.clear(); // Reset stream state so the next puzzle can reuse it
    }
};

/*
 * SudokuApplication class that manages the list of five puzzle files
 * and runs the SudokuSolver on each file sequentially.
 * Authorship: Gemini (original), Claude (puzzle numbering)
 */
```

```
class SudokuApplication {
private:
    std::vector<std::string> puzzleFiles;
    SudokuSolver solver;

public:
    SudokuApplication() {
        puzzleFiles = {
            "puzzle1.txt",
            "puzzle2.txt",
            "puzzle3.txt",
            "puzzle4.txt",
            "puzzle5.txt"
        };
    }

    void run() {
        for (size_t i = 0; i < puzzleFiles.size(); ++i) {
            solver.processPuzzleFile(puzzleFiles[i], static_cast<int>(i) + 1);
        }
    }
};

/*
 * Program entry point.
 * Authorship: Gemini
 */
int main() {
    SudokuApplication app;
    app.run();
    return 0;
}
```

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.

Correctness:

Claude:

Good: runs well and has correct output.

Bad: the new code seems to follow all correctness

Chatgpt:

Good: runs well and has correct output.

Bad:

Execution time:

Claude:

Good: the DFS is still operational and the finding solution codes is untouched so it still runs fast

Bad: I think it runs as fast a I can make it

Chatgpt:

Good: still uses DFS and is still operational

Bad: I think it runs as fast a I can make it

Space complexity:

Claude:

Good: the outputs are not stored and ram and are directly outputted so good on space

Bad: not any that I can think of because the output is directly put on the txt files and not stored

Chatgpt:

Good: the outputs are not stored for the entire run just for when solving is done

Bad:

Maintainability:

Claude:

Good: automatically added comment of where it inserted so it was easy to find changes and insert them into my working file.

It also changed the array initializers from ={false} to ={{false}}, I asked why it did this it said to silence a compiler warning so I think that will help suppress the noise from compiler warnings

Bad: no comments that go into detail of what each line is doing

Chatgpt:

Good:

Bad: some of the definintions are dense with lots, no comments that go into detail of what each line is doing

f. Select one to use as the basis for this assignment and justify why you picked it.

I chose claude code. Both made basically the same changes but I think calude made it more clear where it made changes in the output and so making it easier to update the old file.

For inserting the edits made by calude code I asked for a break down of where it made edits so I could insert it myself into the file. The commot message with this action is “claude code insert”

Then I added my own comment this commit has the message “added line by line comments”

g. Explain what you did to improve it in terms of correctness, execution time,

space complexity, and maintainability.

Correctness: I separated the processPuzzleFile function that was too long and becoming to much of a “master function

Execution time: none

Space complexity: none

Maintainability: I added line by line comments to the code that I changed**.**

3. Make the changes identified in 2g above.

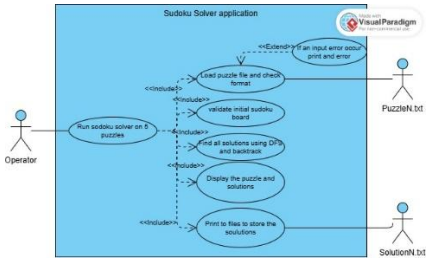
Added line by line comments on code and made changes to the Claude code. I also used claude to help me split the processPuzzleFile function that was too long and becoming to much of a “master function”

4. Compile and debug your program as described in the “C Basics” lecture.

5. Push both your source code, final cycle server executable file, and PDF of your

GenAI analysis to your GitHub repository.

7. Use a GenAI of your choice to modify the UML Use Case Diagram you received for this assignment with the level of detail given in the first UML Use Case



Modeling lecture:

a. Identify/discover a use case

The use case is to run the sudoku solver on 5 puzzles. The primary actor is the operator while solutionN.txt and Puzzle.txt are the secondary actors

b. Provide a brief description for the use case

The operator reads the program that reads each sudoku puzzle iteratively. Then it checks the format of puzzle it gets form PuzzleN.txt. Then I

c. Provide a step-by-step flow (outline) of the events for the use case

1. The program is stated by operator.
2. the system does steps 3 to 8. For each of the 5 puzzles
3. The system opens puzzleN.txt and reads the all cells (this is the include -> Load puzzle file and check format this then if an error extends to stop the initial puzzle.
4. The system then displays the initial puzzle
5. checks the initial clues for duplicates in any row, column, or 3x3 block (this is the include Validate initial Sudoku board).
6. It finds solu using DFS with backtracking. (this os the include: Find all solutions using DFS and backtrack)
7. displays each solution and the total count, or prints the “No solution found,” on (include: Display the puzzle and solutions)
8. The system writes the results to solutionN.txt (include: Print to files to store the solutions).
9. the program ends.